트램 코드 실행 준비: 가상환경 만들기

동아리 트램 캡스톤 · 팀원 공유용 · 2026-11-03 · 대상 코드: `tram_system` (트램_시스템_전체코드.zip)

> **한 줄 요약.** 각자 자기 노트북에서, 코드 폴더 안에 `.venv` 라는 전용 폴더를 한 번 만들고 라이브러리를 거기에 설치합니다. 약 15~30분(인터넷 속도에 따라), 디스크 약 2~5 GB. 다른 사람이 만든 `.venv` 를 복사해 받으면 안 되고, 각자 만들어야 합니다.

0. 가상환경이 뭐고, 왜 필요한가요?

가상환경(`.venv`)은 **이 프로젝트만 쓰는 파이썬 라이브러리 보관함**입니다. OpenCV, YOLO(Ultralytics), PyTorch 같은 큰 라이브러리를 컴퓨터 전체가 아니라 이 폴더에만 깔아서, 다른 수업 과제나 프로그램과 버전이 섞이지 않습니다. 필요 없어지면 `.venv` 폴더만 지우면 깔끔히 정리됩니다.

- 한 번 만들어 두면 다음부터는 **"켜기"(activate) 한 줄**만 하면 됩니다.
- `.venv` 안에는 만든 컴퓨터의 경로가 박혀 있어서, **다른 사람에게 복사·압축해서 넘기면 동작하지 않습니다.** 코드만 공유하고 가상환경은 각자 만드세요.

1. 준비물 (처음 한 번)

항목	방법
Python 3.12 또는 3.13	python.org → Downloads → Windows → **Python 3.12.x**의 "Windows installer (64-bit)". 설치 첫 화면에서 **"Add python.exe to PATH" 체크** 후 Install Now. 먼저 `py -0` 으로 확인해서 **3.12나 3.13이 이미 있으면 새로 깔 필요 없습니다**(이 코드는 3.13에서도 시험함). 둘 다 없을 때만 3.12를 설치하세요. 3.11 이하는 너무 낮고, 3.14는 일부 라이브러리가 아직 지원하지 않을 수 있습니다.
VS Code	code.visualstudio.com에서 설치 → 왼쪽 확장(Extensions, `Ctrl+Shift+X`)에서 **"Python"**(Microsoft) 설치.
코드 폴더	공유받은 `트램_시스템_전체코드.zip` 을 원하는 곳에 압축 풀기 → `tram_system` 폴더가 생깁니다. 경로는 사람마다 달라도 됩니다(예: `C:\capstone\tram_system`, `G:\tram_`). 가능하면 **한글·공백 없는 짧은 경로**를 권합니다. 일부 라이브러리가 한글 경로에서 오류를 내는 경우가 있습니다. Windows "압축 풀기"는 `트램_시스템_전체코드\tram_system` 처럼 한글 폴더를 하나 더 만들기 때문에, 안쪽 `tram_system` 폴더를 꺼내 영어 경로로 옮기는 것을 권합니다.
Visual C++ 재배포 패키지 (오류 날 때만)	PyTorch가 `DLL load failed` 오류를 내면 Microsoft의 "Visual C++ 재배포 가능 패키지 (x64)" 최신 버전(vc_redist.x64.exe)을 설치하고 VS Code를 다시 켭니다.

설치 확인: 아무 PowerShell 창에서 `py -0` 을 치면 설치된 버전 목록이 나옵니다. 목록에 `-V:3.12` 또는 `-V:3.13` 이 보이면 준비 끝입니다.

2. Windows 노트북: VS Code에서 가상환경 만들기

아래 순서는 빈 님 노트북(Windows 11, VS Code 터미널 cmd, Python 3.13, 코드 폴더 `G:\tram_`)에서 실제로 끝까지 해 보고 합격을 확인한 기록을 바탕으로 정리했습니다. 경로는 각자 자기 것으로 바꿔 읽으세요.

1 VS Code에서 코드 폴더 열기

파일 → 폴더 열기 → **코드 폴더**를 선택합니다. 코드 폴더란 안에 `pi_main.py`, `requirements_laptop.txt`, `tools`, `tram`이 **바로** 보이는 폴더입니다. 이름은 `tram_system`이 아니어도 됩니다(예: 빈 님은 내용물을 `G:\tram_`에 바로 복사해서 `G:\tram_`이 코드 폴더). "작성자를 신뢰하시겠습니까?"가 뜨면 신뢰.

주의: 그 안의 `tram` 폴더는 코드 부품을 모아 둔 폴더라서 여기를 열면 안 됩니다.

2 터미널 열고, 종류와 위치 확인

메뉴 터미널 → 새 터미널 (`Ctrl+``). VS Code는 컴퓨터 설정에 따라 둘 중 하나를 엽니다. **둘 다 괜찮고**, 5단계 "켜기" 명령만 다릅니다.

종류	화면 모양	구별법
cmd (명령 프롬프트)	`G:\tram_>`	맨 위에 `Microsoft Windows [Version ...]`가 나오고 줄 앞에 아무것도 없음
PowerShell	`PS G:\tram_>`	줄 앞에 `PS`

위치 확인: 줄 끝 경로가 1단계의 코드 폴더여야 합니다. `G:\tram_\tram>`처럼 한 칸 안쪽이면 `cd ..`로 한 칸 올라갑니다. 다른 폴더면 `cd /d G:\tram_`(cmd) 또는 `cd G:\tram_`(PowerShell)처럼 자기 경로로 이동합니다.

3 설치된 Python 버전 확인

```
py -0
```

예시 결과(빈 님 노트북):

```
 -V:3.13 *        Python 3.13 (64-bit)
 -V:3.10          Python 3.10 (64-bit)
```

목록에 **3.12 또는 3.13**이 있으면 그 숫자를 4단계에 씁니다(위 예시면 3.13). 3.10 이하만 있거나 아무것도 없으면 1장대로 3.12를 설치하고 VS Code를 완전히 껐다 켭니다.

4 가상환경 만들기 (코드 폴더에서 딱 한 번, 10초 정도, 메시지 없음이 정상)

```
py -3.13 -m venv .venv      # 3단계에서 본 숫자로 (3.12면 py -3.12)
```

왼쪽 탐색기에 `.venv` 폴더가 생깁니다. VS Code가 "새 환경을 사용하시겠습니까?"라고 물으면 **예**. 가상환경은 **코드 폴더에 하나만** 있으면 되고, `tram`·`tools` 같은 안쪽 폴더에 따로 만들 필요는 없습니다.

`No suitable Python runtime found`가 나오면 그 버전이 없는 것입니다. 3단계 목록의 숫자로 바꿔 다시 칩니다.

5 가상환경 켜기 (2단계에서 본 터미널 종류에 맞게)

cmd	`.venv\Scripts\activate.bat`

PowerShell	`# 컴퓨터마다 처음 한 번만 (확인을 물으면 Y)`
	`Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`
	`.\.venv\Scripts\Activate.ps1`
	Set-ExecutionPolicy는 Windows가 막아 둔 PowerShell 스크립트 실행을 내 계정에서만 허용하는 설정입니다(관리자 권한 불필요). cmd에서는 필요 없습니다.

줄 맨 앞에 `(.venv)`가 붙어 `(.venv) G:\tram_>`처럼 보이면 성공입니다. 이후 설치·실행은 모두 이 표시가 있는 상태에서 합니다.

6 라이브러리 설치 (10~30분, PyTorch 다운로드가 대부분)

```
python -m pip install --upgrade pip
pip install -r requirements_laptop.txt
```

설치 목록: OpenCV, NumPy, pyzmq(파이↔노트북 통신), Ultralytics(YOLO, PyTorch 포함), OpenVINO(인텔 노트북 가속). 마지막에 `Successfully installed ...`가 나오면 끝입니다. 빨간 `ERROR`가 나오면 3장 문제 해결을 보세요.

7 설치 확인 (자가 시험) (약 20초)

```
python tools\selftest.py
```

합성 영상으로 전체 흐름(정상 → 연결 끊김 예측 → 자체 검출 → 정상)을 돌려 보고, **마지막 줄에 `합격`**이 나오면 설치 완료입니다. 실제로 빈 님 노트북에서 나온 결과(숫자는 조금씩 달라도 됨):

```
물체 | 짝지은 프레임 | 앞 거리 오차 평균(절대) | 상대 오차 | 옆 거리 오차 평균(절대)
  A  |   117 | 0.46 m (치우침 -0.28) | 1.3% | 0.03 m
  B  |    72 | 0.29 m (치우침 -0.18) | 1.1% | 0.04 m
위험 구역 안 프레임 69개 중 '안전'으로 본 프레임: 0개 (목표 0)

모드 순서: LOCAL → ONLINE → PREDICT → LOCAL → ONLINE  O
'위험' 경보가 나왔나: O
위험 구역 안 놓침 0: O

합격
```

읽는 법: 앞 거리 오차 약 1%, 위험 구역 안 물체를 놓친 프레임 0개, 연결이 끊겼을 때 예측(PREDICT) → 파이 자체 검출(LOCAL) → 복구(ONLINE) 순서로 바뀌면 정상입니다. `logs\selftest\` 폴더는 이 시험이 만든 기록이라 지워도 됩니다.

8 VS Code 인터프리터 고르기 (▷ 실행 버튼·자동 완성용)

`Ctrl+Shift+P` → **Python: Select Interpreter** → `.venv`가 붙은 항목(`.\.venv\Scripts\python.exe`) 선택. 오른쪽 아래 상태 표시줄에 `3.13.x ('.venv')`(또는 3.12.x)가 보이면 됩니다. 이렇게 해 두면 새 터미널을 열 때 VS Code가 가상환경을 자동으로 켜 줍니다.

다음에 다시 쓸 때. 설치는 다시 안 합니다. VS Code로 같은 코드 폴더를 열고 터미널에 `(.venv)`가 없으면 켜기 한 줄만 칩니다: cmd는 `.venv\Scripts\activate.bat`, PowerShell은 `.\.venv\Scripts\Activate.ps1`. 끌 때는 `deactivate`.

설치가 끝나면 해 볼 것

```
# 합성 영상 만들고 창으로 보기 (q = 끝)
python tools\make_sample_video.py
python pi_main.py --source sample.mp4 --server local --local color --no-gpio
# 3초에 2.5초 동안 연결이 끊기는 상황
python pi_main.py --source sample.mp4 --server local --local color --no-gpio --drop-at 3 --
drop-for 2.5
```

실제 YOLO, 노트북+파이 연결, 1:10 모형 설정은 코드 폴더의 `README.md` 3~6절에 이어서 있습니다.

3. 자주 나는 문제와 해결

화면에 보이는 것	원인과 해결
`tram` 폴더(또는 다른 안쪽 폴더)에도 가상환경을 따로 만들어야 하나요?	아니요. 코드 폴더의 `.venv` 하나를 모든 코드가 같이 씁니다. 실수로 안쪽에 만들었다면 그 `.venv` 폴더는 지워도 됩니다.
터미널이 `PS`가 아닌데 괜찮나요?	괜찮습니다. cmd(명령 프롬프트)입니다. 켜기만 `.venv\Scripts\activate.bat`로 하면 나머지는 같습니다(2·5단계).
`py` 또는 `python`이(가) 인식되지 않습니다	Python 설치 때 PATH 체크를 안 했거나, 설치 후 VS Code를 다시 안 켠 경우. VS Code를 완전히 껐다 켜고, 그래도 안 되면 Python 설치 파일을 다시 실행 → Modify → "Add Python to environment variables" 체크.
`No suitable Python runtime found` (py -3.12 실행 시)	그 버전이 안 깔린 상태. `py -0`으로 목록을 보고 3.13이 있으면 `py -3.13`으로 바꿔 실행. 둘 다 없으면 1장대로 3.12 설치.
`이 시스템에서 스크립트를 실행할 수 없으므로 ... Activate.ps1 파일을 로드할 수 없습니다`	4단계(ExecutionPolicy)를 안 한 경우. 4단계 실행 후 다시 5단계. 학교 PC처럼 설정을 못 바꾸면, 그 창에서만 임시 허용: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` 후 5단계. 또는 터미널 종류를 cmd(명령 프롬프트)로 바꾸고 `.venv\Scripts\activate.bat`.
`ModuleNotFoundError: No module named 'cv2'` (또는 ultralytics, zmq)	가상환경이 꺼진 채로 실행. 줄 앞에 `(.venv)`가 있는지 확인하고 5단계. ▷ 버튼으로 실행했다면 8단계(인터프리터 선택)를 확인.
`requirements_laptop.txt`를 찾을 수 없음 (No such file)	터미널 위치가 코드 폴더가 아님. 한 단계 위 폴더를 열었을 가능성(zip을 풀면 `tram_system` 폴더가 한 번 더 생김). `cd tram_system`으로 들어가거나 1단계에서 안쪽 폴더를 다시 열기.
`OSError: [WinError 126] ... DLL load failed` (torch 관련)	Visual C++ 재배포 패키지가 없음. 1장 표의 마지막 줄대로 설치 후 VS Code 재시작.
설치 중 `Read timed out`, 아주 느림	네트워크 문제. 같은 명령을 다시 실행하면 받은 것은 건너뛰고 이어서 받습니다. 학교 와이파이가 막히면 휴대폰 핫스팟으로 시도.

`No space left on device`, 디스크 부족	PyTorch 포함 2~5 GB가 필요. 공간 정리 후 `.venv` 폴더를 지우고 3단계부터 다시.
외장 SSD/USB에서 잘 되던 가상환경이 갑자기 안 됨 (`Fatal error in launcher`, `did not find executable`)	드라이브 문자(G:, F: 등)가 바뀐 경우. `.venv`는 만든 당시 경로를 기억하므로 **`.venv` 폴더를 지우고 3·5·6단계를 다시** 하면 됩니다(코드와 결과 파일은 그대로). 폴더를 다른 곳으로 옮겼을 때도 같습니다.
뭔가 꼬여서 처음부터 하고 싶음	VS Code 터미널에서 `deactivate` → 탐색기에서 `.venv` 폴더 삭제 → 3단계부터. 코드는 건드리지 않습니다.

4. 라즈베리파이 5 (Raspberry Pi OS 64-bit)

파이는 카메라(picamera2)와 부저·LED(gpiozero)를 OS에 들어 있는 것으로 써야 해서, 가상환경을 **`--system-site-packages`** 옵션으로 만드는 것이 핵심입니다. 최신 Raspberry Pi OS는 가상환경 밖에서 `pip install`을 막아 두므로 (`externally-managed-environment` 오류) 가상환경은 필수입니다.

```
# 1) OS 쪽 부품 설치 (처음 한 번)
sudo apt update && sudo apt install -y python3-picamera2 python3-opencv python3-gpiozero

# 2) 코드 폴더를 파이에 복사한 뒤 (USB 또는 노트북에서: scp -r tram_system pi@파이IP:~)
cd ~/tram_system
python3 -m venv --system-site-packages .venv      # 옵션 빼먹으면 카메라가 안 보임
source .venv/bin/activate                         # 줄 앞에 (.venv)
pip install -r requirements_pi.txt

# 3) 점검: 카메라 사진 1장, 부저·LED 켜 보기, NCNN 모델 속도
python tools/check_env.py --picam --gpio --ncnn yolo11n_ncnn_model
```

- 다음부터는 `cd ~/tram_system` → `source .venv/bin/activate`만 하면 됩니다.
- `numpy.dtype size changed` 오류: OS의 NumPy와 pip NumPy가 섞인 경우. `pip install "numpy<2"` 후 다시.
- `yolo11n_ncnn_model` 폴더는 노트북에서 `python tools\export_models.py --target all`로 만든 뒤 파이로 함께 복사합니다.
- 노트북의 `.venv`를 파이로 복사하면 안 됩니다(운영체제가 달라 동작하지 않음). 파이에서 새로 만드세요.

5. 공유할 때 지킬 것

RailSem19 데이터는 복사해서 나눠 주면 안 됩니다. 실제 선로 사진 실험(`tools\rail_photo_test.py`, `tools\rail_viewer.py`)에 쓰는 RailSem19 사진·라벨은 데이터셋 사이트에 **각자 등록해서 받은 사람만** 비상업 연구용으로 쓸 수 있습니다. 이 실험을 하려는 팀원은 각자 신청해서 받고, 실행할 때 `--data` 뒤에 자기 컴퓨터의 폴더 경로를 넣으세요. 이 스크립트로 만든 결과 그림도 팀 밖·저장소에 올리지 말고, 발표에 쓸 때는 얼굴·번호판을 가립니다.

- **공유해도 되는 것:** 코드 폴더(`.py`, `README.md`, `requirements_*.txt`, 설정 JSON), 이 안내서.
- **공유하지 말 것:** `.venv` 폴더(각자 만듦), RailSem19 데이터와 그 결과 그림, `rail_test_out\`.
- **올리면 용량만 큰 것:** `yolo11n.pt`·모델 변환 폴더(각자 `export_models.py`로 생성 가능), `logs\`, 영상 파일.
- 명령 예시에 나오는 경로(`G:\rail_pic` 등)는 예시일 뿐입니다. 각자 자기 경로로 바꿔서 쓰세요.

- 나중에 GitHub 저장소를 만들면 `.gitignore`에 `.venv/`, `logs/`, `rail_test_out/`, `*.pt`, `*_openvino_model/`, `*_ncnn_model/`를 넣어 두면 실수로 올라가지 않습니다.

6. 한눈에 보는 순서 (Windows)

```
# 코드 폴더(pi_main.py가 보이는 곳)를 VS Code로 열고, 터미널(Ctrl+`)에서
py -0                                          # 3.12 또는 3.13이 있는지 확인
py -3.13 -m venv .venv                         # 위 숫자로 (한 번만)
.venv\Scripts\activate.bat                      # cmd일 때. 줄 앞에 (.venv) 확인
python -m pip install --upgrade pip
pip install -r requirements_laptop.txt         # 10~30분
python tools\selftest.py                        # 마지막 줄 '합격'
# Ctrl+Shift+P → Python: Select Interpreter → .venv

# PowerShell(줄 앞에 PS)이면 activate 줄 대신:
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned   # 처음 한 번, Y
.\.venv\Scripts\Activate.ps1
```